everfit

Mobile Test Guide

Thank you for your interest in joining **Everfit**!

As part of our hiring process, we'd like you to complete a short **assignment** to help us understand your **skills, problem-solving approach, and attention to detail**.

🧭 Purpose of the Test

This test assesses your ability to:

- Convert a design into pixel-perfect UI
- Write clean, maintainable, and modular code
- Deliver complete results efficiently

Overview

Build out a calendar to manage working training for users. The app should allow users to:

1. View the title and status of a week's workouts.
2. Mark workouts as completed (locally).
   - **Figma Design:** https://www.figma.com/design/APChc5i8CKTSn7CZg7Gg36/Everfit?node-id=23776-49540
   - **Mock API Endpoint:** https://mock.internalef.com/workouts

🤖 AI & Tooling Policy

We embrace modern development tools and want to see how you leverage them to increase your productivity! You are highly encouraged to use AI assistants (ChatGPT, Claude, GitHub Copilot, Cursor, etc.).

However, you must own and understand 100% of the code you submit. To help us evaluate your AI collaboration skills, please adhere to the following:

1. Transparency (Required)

- Create a section in your README.md named **"AI Collaboration"**.
- List the tools you used (e.g., "Cursor for IDE, Claude 3.5 Sonnet for logic").
- Share **2-3 specific prompts** you used to solve non-trivial parts of this test (e.g., generating the UI layout for the calendar, or writing the caching logic). We want to see your prompt engineering skills!

2. Best Practices for Using AI

everfit

- ✅ **DO use AI for:** Generating boilerplate UI code, scaffolding unit tests, parsing JSON models, or researching architectural patterns.
- ❌ **DON'T use AI to:** Blindly generate the entire project without understanding the data flow.
- 📝 **Code Comments:** If AI generated a complex piece of logic (like date math for the calendar view), leave a brief comment in your code: `// Generated via ChatGPT. Adjusted to handle edge case X.`

3. The Video Walkthrough (Mandatory)

- Please record a short **3-5 minute video** (using Loom, Zoom, or screen record) demoing the app.
- **Walk us through your code:** Explain your architecture (e.g., why you chose MVVM-C), how the data flows from the mock API to the UI, and how you implemented caching. If you used AI, mention how it sped up your workflow.
- *Note: In a follow-up interview, we may ask you to explain specific blocks of code. If you cannot explain code that AI generated for you, it will be considered a red flag.*

Product Requirements

Training Calendar Screen

1. Week View (Monday to Sunday)

- The screen should show 1 week of day containers.
- The day of the week (Mon, Tue, etc.) should show up on the top left of each cell.
- The day of the month for this current week should show up just below the day of the week.
- The date for "Today" should be highlighted in purple (as shown in the design).

2. Workout Containers

- Each day container can hold multiple workouts.
- Each workout container must display:
  - Workout name (truncate with "..." if the name is too long).
  - Total number of exercises belonging to that workout.
  - Status indicator. Statuses include:
    - Past: *Missed* or *Completed*
    - Today: *Assigned* or *Completed*
    - Future: *(greyed out, no specific text needed)*
  - Colors should dynamically reflect these states according to the provided design.

3. Interactivity

everfit

- When tapping on a non-empty cell (a workout), it should toggle the mark/unmark state locally (based on the item ID).
- If marked as completed, a checkmark icon should appear on the right side of the cell.

4. Loading State

- When the API request is loading, all day cells should still display the correct dates, but show an empty/loading state for the data.

Technical Requirements

1. **Architecture:** Use a modern, clean architecture that you are confident in (e.g., MVVM, MVVM-C). Keep it scalable and modular.
2. **Networking:** Fetch data from a mock API endpoint or a mock JSON file (provided in the repo).
    a. **For iOS applicants**: Please use modern Swift Concurrency (async/await) for data fetching.
    b. **For Android applicants**: Please use Kotlin Coroutines / Flow / RxAndroid.
    c. **Mock API Endpoint:** http://demo6732818.mockable.io/workouts
3. **Persistence:** The data must be cached locally. When the user opens the app the next time, it should load from the cache immediately while fetching updates (if any). You may use:
    a. **iOS:** Realm, CoreData, SwiftData, or any persistence layer you prefer.`
    b. **Android**: Room, DataStore, or preferred persistence layer.
4. **UI Framework:**
    a. **iOS:** You are free to use UIKit (programmatic preferred) or SwiftUI.
    b. Android: Jetpack Compose or XML layouts with ViewBinding/DataBinding.
5. **Design Asset:** Refer to the Figma design:
    – https://www.figma.com/design/APChc5i8CKTSn7CZg7Gg36/Everfit?node-id=23776-49540

🌟 Bonus Points

We would love to learn more about your holistic engineering approach!

In your video walkthrough (or README.md), please briefly describe the **full end-to-end process** you typically take when you receive a new feature request—from reading the initial requirements to releasing it to users.

Candidates who can demonstrate a mature, well-rounded development workflow will earn bonus points.

⏰ **Time Limit** Please complete this test within **24 hours from the time you receive this email**.

📩 Submission Instructions

When you complete the test, please:

1. Push your code to a public GitHub repository.

2. Ensure your README.md contains build instructions, your AI usage notes, and the link to your video walkthrough.
3. Share the repository link with us.

Thank you again for your time and effort - we look forward to reviewing your work! Good luck!